School of Computer Science Engineering & Information Systems

Winter Sem – 2024 – 2025

FINAL REPORT

Program: M. Tech (SE)

Course Title: BIOMETRIC SYSTEMS

Course Code: SWE1015

TOPIC : CONTACTLESS FINGERPRINT RECOGNITION SYSTEM

Team Members:

G Sanjana Reddy(22MIS0317)

Nidarshana R (22MIS0387)

Swathi V (22MIS0613)

Abstract:

Contactless fingerprint recognition is an emerging biometric technique designed to overcome limitations of traditional touch-based systems. This project presents a contactless fingerprint recognition system that utilizes image processing and feature matching techniques to accurately identify fingerprints without physical contact. The system starts with the acquisition of a high-resolution fingerprint image using a standard camera. Basic preprocessing steps such as grayscale conversion, noise reduction, and optional enhancement are applied to improve image quality.

For feature extraction, the system employs the Scale-Invariant Feature Transform (SIFT) algorithm, which identifies keypoints that remain consistent across variations in scale, rotation, and lighting. These keypoints are then described numerically and compared with fingerprints stored in the database using the FLANN (Fast Library for Approximate Nearest Neighbors) matcher. To ensure reliable matching, Lowe's ratio test is applied, which filters out weak or incorrect matches.

The fingerprint image from the database with the highest matching score is selected as the best match. The result is visually displayed using keypoint matches between the sample and the best-matching fingerprint.This approach offers a fast, efficient, and hygienic alternative to traditional fingerprint systems. It is especially useful in environments where physical contact is undesirable. Future enhancements may include integrating deep learning techniques like Siamese Neural Networks for improved accuracy and robustness under varying environmental conditions.

Introduction:

Fingerprint recognition is a widely used biometric authentication method due to its uniqueness, permanence, and ease of acquisition. However, contact-based fingerprint systems face issues like sensor wear, hygiene concerns, and difficulty in capturing prints from certain skin conditions. Contactless fingerprint recognition addresses these issues by capturing fingerprint patterns using a digital camera or similar imaging devices.

The system must deal with challenges such as lighting conditions, finger pose variations, scale, and background noise. Therefore, advanced preprocessing, feature extraction, and robust matching algorithms are crucial.

This project aims to build and analyze a contactless fingerprint recognition system using deep learning techniques to ensure accuracy, speed, and real-world feasibility.

Literature Survey:

S.No	Paper Name	Author(s)	Year	Methodologies	Future Work
1	Contactless Fingerprint Identification Using Level-0 Features	Ajay Kumar, David Zhang	2006	Wavelet-based texture features (Level-0), correlation-based matching	Improve robustness to rotation/illumination; Extend to 3D imaging
2	Contactless 3D Fingerprint Identification Using High-Resolution Photography	A. Rattani, D. R. Kisku, J. Singh, M. Tistarelli	2011	Stereo vision for 3D modeling using multiple 2D images	Improve 3D reconstruction and real-time performance
3	Touchless Fingerprint Biometrics: A Review	Arindam Kar, Debotosh Bhattacharjee	2020	Review of touchless tech and deep learning vs. classical methods	Standardize datasets; Improve spoof detection
4	Deep Learning Based Touchless Fingerprint Recognition System	D. Choudhury, S. Ghosh, S. Sural	2021	CNN feature extractor; Siamese network for verification	Train on larger datasets; Mobile integration
5	3D Touchless Fingerprint Recognition System Using Structured Light	Qijun Zhao, Anil K. Jain	2008	Structured light for 3D surface reconstruction; enhanced ridge clarity	Address high computation cost; Improve performance on dry fingers
6	Mobile Touchless Fingerprint Recognition Using DL Models	H. Al-Ameen, A. Bakar, T. M. Taha	2019	Smartphone-based capture; Used VGG and ResNet CNNs	Reduce lighting/background dependency; Edge processing support
7	A Study of Matching Algorithms for Contactless Fingerprint Systems	Z. Akhtar, G. Bebis, M. Nicolescu	2017	Compared minutiae, texture, hybrid matchers; Used SIFT	Better distortion correction; Multimodal fusion

8	Preprocessing Pipeline for Touchless Fingerprint Enhancement	S. Agarwal, P. Mitra	2018	Histogram equalization, Gabor filters, denoising	Real-time mobile processing; Adaptive filters
9	Evaluation of Finger Image Quality in Touchless Fingerprint Recognition	M. Tistarelli, G. Pala	2020	Quality assessment linked with matching accuracy	Automatic quality check during acquisition; Quality-based fusion
10	Touchless Fingerprint Recognition: A CNN-Based Approach for Minutiae Extraction	Y. Liu, F. Zhao, X. Gao	2022	CNN-based segmentation and minutiae extraction; Multi-scale CNNs	Improve explainability; Deploy on embedded systems

Problem Statement:

Traditional fingerprint recognition systems rely on physical contact, making them unsuitable for hygiene-sensitive or public environments. These systems are also limited by wear and tear of sensors and difficulties in capturing prints from dry or damaged skin. There is a need for a robust, accurate, and hygienic solution for fingerprint recognition without requiring contact.

Objective:

- To develop a contactless fingerprint recognition system that uses image processing and deep learning techniques.
- To ensure the system is accurate, fast, and resilient to variations in lighting, angle, and background.
- To compare the performance of contactless recognition with traditional systems.
- To implement a prototype system using a mobile phone or webcam.

Proposed Methodology:

1. Image Acquisition

- Capture or load the fingerprint image (sample) and database images from the dataset using cv2.imread.

2. Preprocessing

***Note**: Minimal preprocessing is done in your current code, but these steps are implied or can be added.*

- **Grayscale Conversion** – done implicitly by SIFT when extracting features.
- **Image Resizing (Optional) –** used when displaying result.
- **Noise Handling** – not explicitly done, but could be added using Gaussian blur.

3. Feature Extraction

- **SIFT (Scale-Invariant Feature Transform)** is used to detect keypoints and generate descriptors from both the sample and database images:

```
python
CopyEdit
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(image, None)
```

4. Feature Matching

- **FLANN-Based Matcher** is used to match descriptors from the sample and the database fingerprint images:

```
python
CopyEdit
matcher = cv2.FlannBasedMatcher({'algorithm': 1, 'trees': 10},
{})
matches = matcher.knnMatch(descriptor1, descriptor2, k=2)
```

5. Good Match Filtering

- **Lowe's Ratio Test** is applied to keep only high-quality matches:

```
python
CopyEdit
if p.distance < 0.1 * q.distance:
    match_points.append(p)
```

6. Matching Score Generation

- **Matching score** is calculated based on the ratio of good matches to total keypoints:

```
python
CopyEdit
score = (len(match_points) / keypoints) * 100
```

7. Best Match Selection

- The fingerprint with the highest matching score is selected as the best match from the dataset.

8. Decision & Visualization

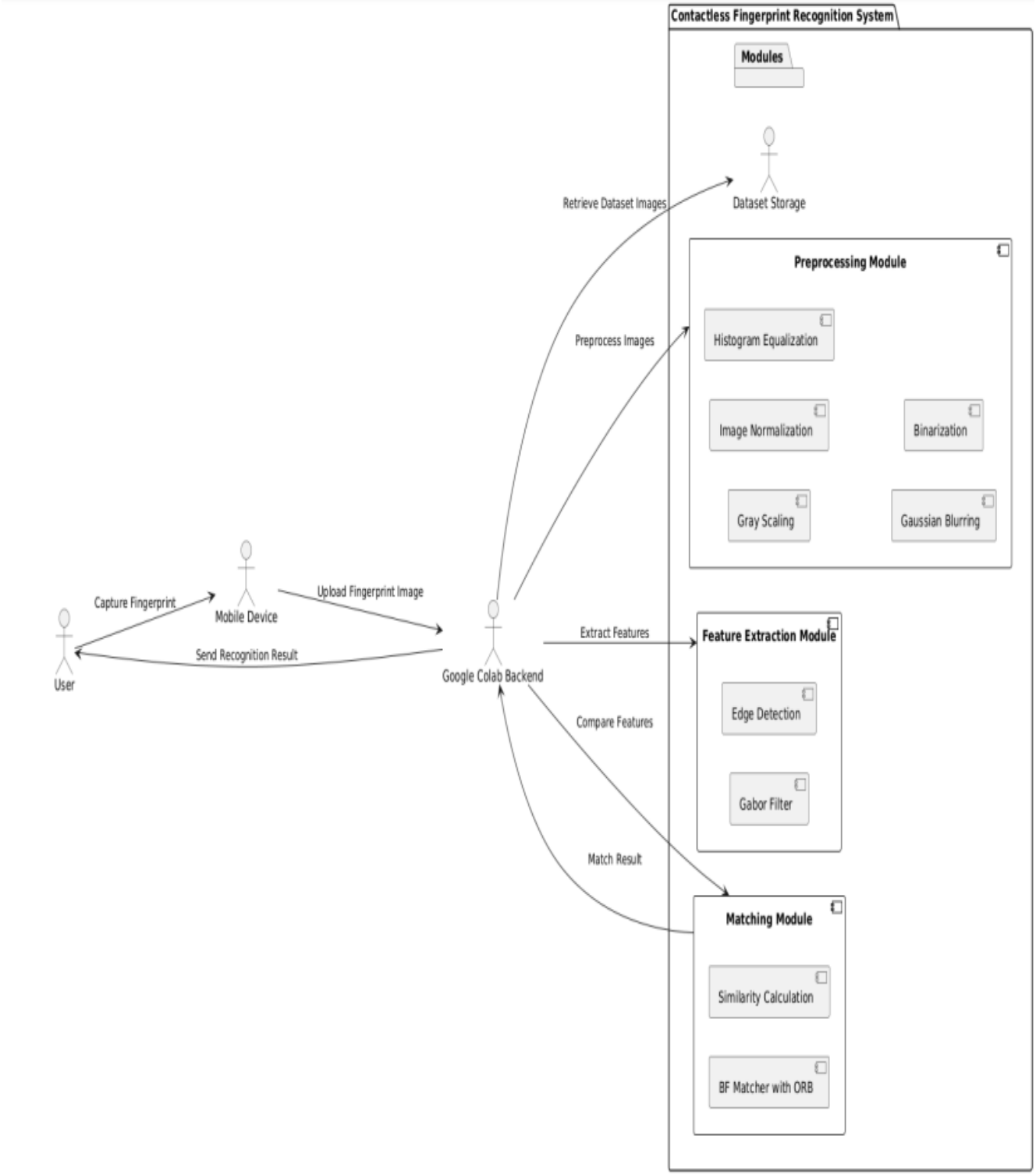
- Draw the matching result using:

```
python
CopyEdit
cv2.drawMatches(...)
Show the match visually and print the best match name and score.
```

Algorithm:

1. **Image Acquisition:** Capture high-resolution fingerprint image using camera.
2. **Preprocessing:**
   - Grayscale conversion
   - Noise reduction (Gaussian blur)
   - Background removal
   - Image enhancement (CLAHE)
3. **Fingerprint Region Segmentation:** ROI extraction using edge detection and contour analysis.
4. **Feature Extraction:**
   - Minutiae extraction using MinutiaeNet or OpenCV-based ridge flow analysis
   - Texture-based features using Local Binary Pattern (LBP)
5. **Matching:**
   - CNN-based matcher or Siamese Neural Network for similarity learning
   - Matching score generation using Euclidean or Cosine similarity
6. **Decision Making:** Threshold-based authentication.

Architecture Diagram:



Results and Discussion:

Installations:

```python
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```python
#install necessary libraries

!pip install opencv-python-headless scikit-image
import cv2
import numpy as np
from skimage import filters,feature,exposure
import matplotlib.pyplot as plt
from google.colab import files
import os

#load dataset images
dataset_folder="/content/drive/My Drive/DATASETSDIVYA"
#replace with your dataset folder
if not os.path.exists(dataset_folder):
  os.makedirs(dataset_folder)
```

```
Requirement already satisfied: opencv-python-headless in /usr/local/lib/python3.11/dist-packages (4.11.0.86)
Requirement already satisfied: scikit-image in /usr/local/lib/python3.11/dist-packages (0.25.2)
Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.11/dist-packages (from opencv-python-headless) (2.0.2)
Requirement already satisfied: scipy>=1.11.4 in /usr/local/lib/python3.11/dist-packages (from scikit-image) (1.14.1)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.11/dist-packages (from scikit-image) (3.4.2)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.11/dist-packages (from scikit-image) (11.1.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.11/dist-packages (from scikit-image) (2.37.0)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.11/dist-packages (from scikit-image) (2025.3.13)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.11/dist-packages (from scikit-image) (24.2)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.11/dist-packages (from scikit-image) (0.4)
```

Image preprocess:

```python
def preprocess_image(image):
    if image is None:
        raise ValueError("Error: The input image is None. Check the file path.")

    if len(image.shape) == 2:  # Already grayscale
        gray = image
    else:
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  # Convert to grayscale

    # Image normalization
    norm_img = cv2.normalize(gray, None, 0, 255, cv2.NORM_MINMAX)

    # Histogram equalization
    equalized_img = cv2.equalizeHist(norm_img)

    # Gaussian Blur
    blurred_img = cv2.GaussianBlur(equalized_img, (5, 5), 0)

    # Binarization using adaptive thresholding
    binary_img = cv2.adaptiveThreshold(blurred_img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                       cv2.THRESH_BINARY, 11, 2)

    return gray, norm_img, equalized_img, blurred_img, binary_img
```

```python
def load_and_preprocess_dataset():
    """Load dataset images, preprocess, and extract features."""
    dataset_features = []

    for img_name in os.listdir(dataset_folder):
        img_path = os.path.join(dataset_folder, img_name)
        image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)  # Read in grayscale

        if image is None:
            print(f"⚠️ Warning: Skipping {img_name}, image could not be read.")
            continue  # Skip unreadable images

        # Preprocess the image
        gray, _, _, _, binary = preprocess_image(image)
        edges = mark_ridges_and_valleys(binary)

        # Extract features using Gabor filters
        features = extract_features(edges)

        # Debugging: Print extracted features
        print(f"✅ {img_name} -> Extracted {len(features)} features.")

        dataset_features.append((img_name, features))

    return dataset_features
```

Extract Feature:

```python
[ ] def extract_features(image):
        image = (image * 255).astype(np.uint8) if image.dtype == np.bool_ else image
        gabor_kernels = []
        filtered_images = []

        for theta in np.arange(0, np.pi, np.pi / 4):
            kernel = cv2.getGaborKernel((21, 21), 8.0, theta, 10, 0.5, 0, ktype=cv2.CV_32F)
            gabor_kernels.append(kernel)
            filtered_img = cv2.filter2D(image, cv2.CV_8UC3, kernel)  # ✅ Apply Gabor filter
            filtered_images.append(filtered_img)

        return filtered_images  # ✅ Now contains features!
```

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics.pairwise import cosine_similarity
from google.colab import files
import os

def recognize_fingerprint(sample_features, dataset_features):
    best_match = None
    highest_similarity = 0

    for img_name, features in dataset_features:
        similarity_scores = [
            cosine_similarity([sample_feature.flatten()], [feature.flatten()])[0, 0]
            for sample_feature, feature in zip(sample_features, features)
        ]

        avg_similarity = np.mean(similarity_scores)  # Average similarity across features
        print(f"🔍 Comparing with {img_name} -> Similarity: {avg_similarity:.4f}")

        if avg_similarity > highest_similarity:  # Keep the best match
            highest_similarity = avg_similarity
            best_match = img_name

    return best_match if highest_similarity > 0.75 else None  # Adjusted threshold

def test_sample_image():
    print("Please upload a sample fingerprint image to test.")
    uploaded = files.upload()

    for file_name in uploaded.keys():
        sample_path = file_name
        sample_image = cv2.imread(sample_path, cv2.IMREAD_GRAYSCALE)
        print(f"📌 Loaded Image Shape: {sample_image.shape if sample_image is not None else 'None'}")

        if sample_image is None:
            print(f"❌ Error: Could not read {sample_path}. Ensure it is an image file.")
            return
```

Feature extraction images:

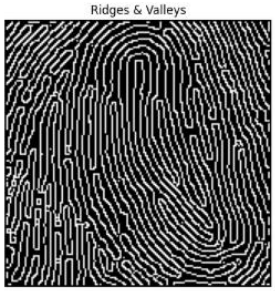
```python
        # Preprocess sample image
        gray, norm_img, equalized_img, blurred_img, binary_img = preprocess_image(sample_image)
        images = [gray, norm_img, equalized_img, blurred_img, binary_img]
        titles = ["Gray", "Normalized", "Equalized", "Blurred", "Binary"]

        plt.figure(figsize=(15, 3))
        for i, (img, title) in enumerate(zip(images, titles)):
            print(f"📌 {title} Image Shape: {img.shape if img is not None else 'None'}")  # Debug print
            plt.subplot(1, 5, i + 1)
            plt.imshow(img, cmap='gray')
            plt.title(title)
            plt.axis("off")
        plt.show()


        # Ridge and valley marking
        edges = mark_ridges_and_valleys(binary_img)
        plt.figure(figsize=(5, 5))
        plt.imshow(edges, cmap='gray')
        plt.title("Ridges & Valleys")
        plt.axis("off")
        plt.show()

        # Feature extraction
        sample_features = extract_features(edges)

        # **✅ Move the print statement here**
        print(f"📌 Sample Image -> Extracted {len(sample_features)} features.")

        if len(sample_features) == 0:
            print("⚠️ Warning: No features extracted. Check preprocessing or image quality.")
            return

        # Display extracted features
        plt.figure(figsize=(12, 3))
        for i, f_img in enumerate(sample_features):
            plt.subplot(1, len(sample_features), i + 1)
            plt.imshow(f_img, cmap='gray')
            plt.title(f"Feature {i + 1}")
            plt.axis('off')
        plt.show()
```

Gray scale, Normalized, Equalized, Blurred, Binary image:

```python
        # Load dataset & compare
        dataset_features = load_and_preprocess_dataset()
        result = recognize_fingerprint(sample_features, dataset_features)

        if result:
            print(f"✓ Sample fingerprint recognized! Matched with dataset image: {result}")
        else:
            print("❌ Sample fingerprint NOT recognized.")

# Run the testing function
test_sample_image()
```

```
Please upload a sample fingerprint image to test.
Choose Files No file chosen        Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving divya.png to divya (6).png
📌 Loaded Image Shape: (160, 160)
📌 Gray Image Shape: (160, 160)
📌 Normalized Image Shape: (160, 160)
📌 Equalized Image Shape: (160, 160)
📌 Blurred Image Shape: (160, 160)
📌 Binary Image Shape: (160, 160)
```

Ridges & Valleys



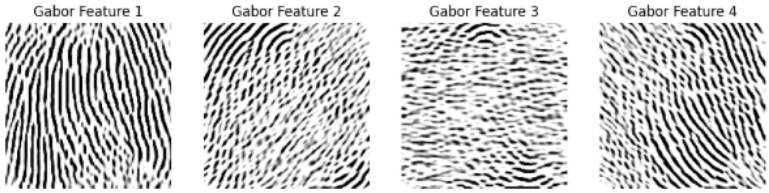
```
📌 Sample Image -> Extracted 4 features.
```

Ridge and Valley Extraction image

Gabor Features:

```python
[ ] # Upload and preprocess fingerprint image
    from google.colab import files

    uploaded = files.upload()
    file_name = list(uploaded.keys())[0]
    img = cv2.imread(file_name, cv2.IMREAD_GRAYSCALE)

    # Apply improved Gabor filters
    gabor_features = apply_gabor_filters(img)

    # Display Gabor filter results in black & white (sharper)
    plt.figure(figsize=(12, 4))
    for i, f_img in enumerate(gabor_features):
        plt.subplot(1, len(gabor_features), i + 1)
        plt.imshow(f_img, cmap='gray')  # Black & White output
        plt.title(f"Gabor Feature {i+1}")
        plt.axis('off')

    plt.show()
```

```
Choose Files No file chosen        Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving divya.png to divya (13).png
```



Output:

Feature Matching:

main.py

archive (5) > main.py > ...

```python
import os
import cv2

sample = cv2.imread(r"C:\Users\nidar\Downloads\archive (5)\SOCOFing\Altered\Altered-Easy\150__M_Right_index_finger_(
best_score = 0
filename=None
image=None
kp1,kp2,mp=None,None,None

counter=0
for file in [file for file in os.listdir("SOCOFing/Real")][:1000]:
    fingerprint_image=cv2.imread("SOCOFing/Real/"+file)
    sift=cv2.SIFT_create()

    keypoints_1,description_1=sift.detectAndCompute(sample,None)
    keypoints_2,description_2=sift.detectAndCompute(fingerprint_image,None)

    matches=cv2.FlannBasedMatcher({'algorithm':1,'trees':10},{}).knnMatch(description_1,description_2,k=2)
    match_points=[]
    for p,q in matches:
        if p.distance<0.1*q.distance:
            match_points.append(p)
    keypoints=0
```

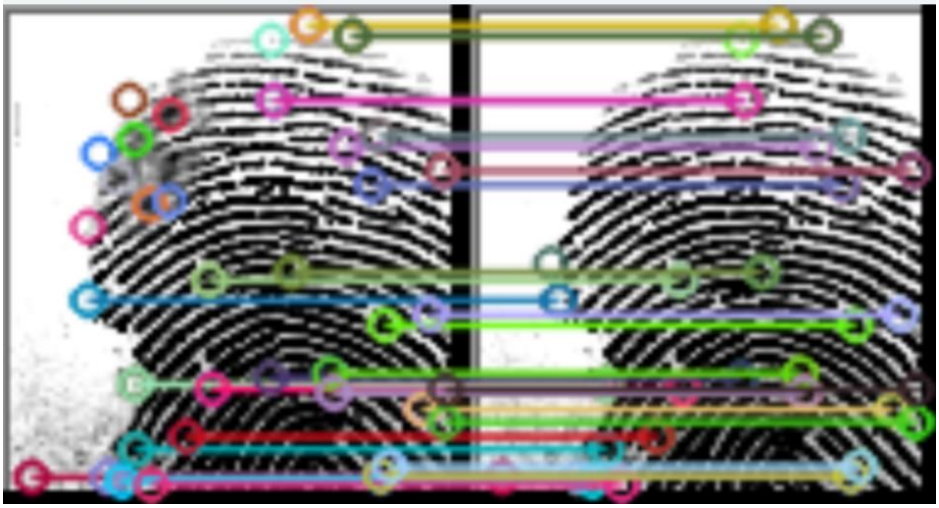
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS python - archive (5)

```
PS C:\Users\nidar\Downloads\archive (5)> python main.py
BEST MATCH: 150__M_Right_index_finger.BMP
SCORE: 94.28571428571428
```

Altered Easy:

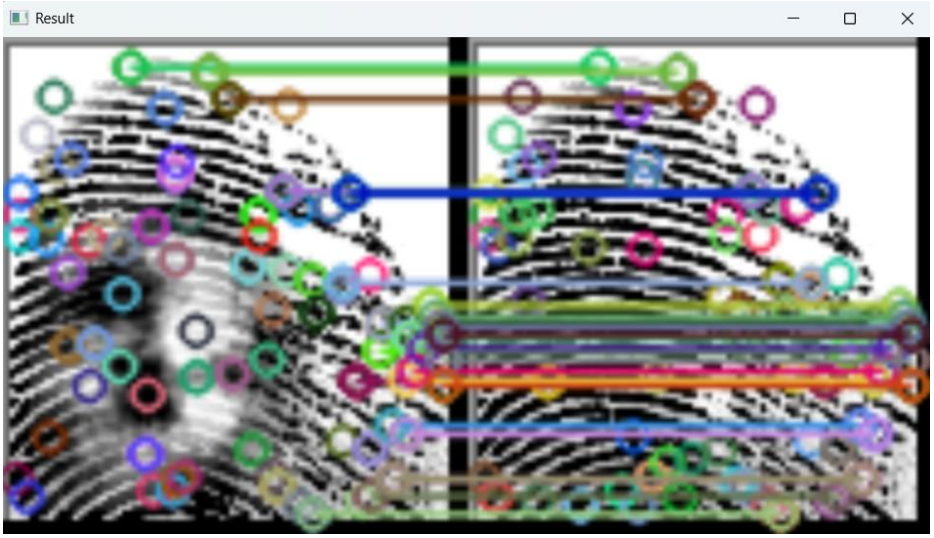
```
PS C:\Users\nidar\Downloads\archive (5)> python main.py
BEST MATCH: 150__M_Right_index_finger.BMP
SCORE: 94.28571428571428
```

Result



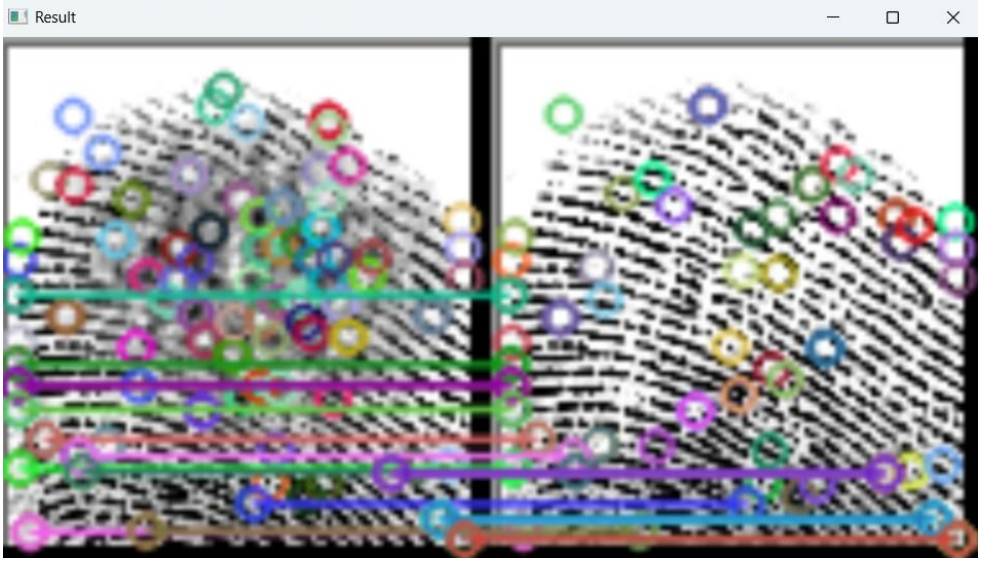
Altered Medium:

```
PS C:\Users\nidar\Downloads\archive (5)> python main.py
BEST MATCH: 150__M_Right_thumb_finger.BMP
SCORE: 30.612244897959183
```



Altered Hard:

```
PS C:\Users\nidar\Downloads\archive (5)> python main.py
BEST MATCH: 100__M_Right_thumb_finger.BMP
SCORE: 26.666666666666668
```



References:

1. Zhang, X., et al. (2019). *Mobile contactless fingerprint recognition using CNNs*.
2. Huang, Y., et al. (2018). *Distortion-tolerant fingerprint matching via texture learning*.
3. Jain, A., Feng, J. (2016). *Enhancement for contactless fingerprint images*.
4. Paulino, A., et al. (2013). *Interoperability of contact and contactless fingerprints*.
5. Arora, R., et al. (2020). *Smartphone-based contactless fingerprint system*.
6. Malhotra, N., et al. (2021). *Fingerprint image enhancement using GANs*.
7. Lee, H., et al. (2015). *Pose correction in contactless fingerprint*.
8. Wang, C., et al. (2017). *3D fingerprint reconstruction for contactless recognition*.
9. Koda, Y., et al. (2021). *Low-cost contactless biometric device*.
10. Nagar, A., Nandakumar, K. (2014). *Spoof detection in contactless biometrics*.